

BibOps

Banc d'évaluation LLM pour le support IT Michelin

Rapport final – Projet Ensimag x Michelin

Auteur : Mohammed Akram Lrhorfi

Partenaire industriel : Michelin

Ecole : Ensimag

Depot analyse : BibOps-michelin-ensimag-againux

Date : 12 mai 2026

Ce rapport est genere a partir du depot local et des artefacts produits par les scripts de benchmark. Les chiffres cites proviennent en particulier de `data/outputs/benchmark/comparison_results.json`, `data/outputs/benchmark/security_race_report.json`, `data/outputs/benchmark/benchmark_mcp.json` et des sorties de tests unitaires executees localement.

Table des matières

Résumé	1
1 Introduction	3
1.1 Contexte industriel	3
1.2 Objectifs du projet	3
1.3 Perimetre du rapport	4
1.4 Apports principaux	4
2 Description conceptuelle du problème	6
2.1 Nature d'un ticket de support IT	6
2.2 Limites d'un LLM unique	6
2.2.1 Absence d'etat externe	6
2.2.2 Risque d'hallucination procedural	7
2.2.3 Evaluation trompeuse par fluidite	7
2.3 Interet et complexite d'un systeme agentique	7
2.4 Probleme d'evaluation	7
2.5 Risques specifiques aux agents connectes	8
2.6 Contraintes de conception	8
3 Présentation de la solution	10
3.1 Vue d'ensemble	10
3.2 Agent de support IT	11
3.2.1 Contrat de decision	11
3.2.2 Routage et memoire	11
3.2.3 Outils disponibles	12
3.3 Moteur d'evaluation	13
3.3.1 Qualite	13
3.3.2 Securite	13
3.3.3 Politique composite	13
3.4 Pipelines de benchmark	14
3.5 Racing Arena	15
3.5.1 Hub FastAPI et flux SSE	15
3.5.2 Equipes et graphes LangGraph	15
3.6 Traces et reproductibilite	15

4	Résultats	17
4.1	Conditions experimentales	17
4.2	Comparaison LLM Unique vs Système Multi-Agents	18
4.2.1	Analyse du score composite	20
4.3	Qualite des outils et MCP	21
4.4	Tests A/B et biais de position	22
4.5	Modeles zero-shot et scores historiques	23
4.6	Racing Arena adversariale	24
4.7	A2A et Kaggle	25
4.8	Verification logicielle	26
4.9	Synthese des resultats	26
5	Discussion	28
5.1	Interpretation du resultat principal	28
5.2	Pourquoi la qualite reste faible	28
5.2.1	Modele local limite	28
5.2.2	Corpus et references incomplets	28
5.2.3	Prompts et format de sortie	28
5.3	Apport de la politique composite	29
5.4	Robustesse et securite agentique	29
5.5	Limites des benchmarks	29
5.5.1	Taille des echantillons	29
5.5.2	Dependance aux juges LLM	30
5.5.3	Comparabilite des artefacts	30
5.5.4	Benchmark A2A non concluant	30
5.6	Qualite logicielle	30
5.7	Perspectives d'amelioration	30
5.8	Positionnement general	31
6	Conclusion	32

Table des figures

3.1	Vue d'ensemble de l'architecture BIBOPS.	10
3.2	Boucle ReAct de l'agent IT.	12
3.3	Poids de la politique composite.	14
3.4	Flux de donnees de la Racing Arena.	15
4.1	Comparaison globale des deux architectures	18
4.2	Synthese du coeur de benchmark	20
4.3	Verdicts composites	20
4.4	Benchmark MCP	21
4.5	Resultat du test A/B LLM	22
4.6	Test de biais de position	22
4.7	Scores par modele	23
4.8	Latence et score	24
4.9	Synthese Racing Arena	24
4.10	Rapport de securite Racing	25
4.11	Synthese finale des benchmarks	26

Liste des tableaux

1	Synthese du benchmark principal extrait de <code>comparison_results.json</code>	2
2.1	Ponderation de <code>CompositePolicy</code>	8
3.1	Organisation fonctionnelle du depot.	11
3.2	Politiques d’execution des outils de support.	12
4.1	Sources chiffrees exploitees.	17
4.2	Resultats principaux extraits de <code>comparison_results.json</code>	19
4.3	Composantes normalisees de la politique composite.	21
4.4	Synthese des benchmarks MCP.	21
4.5	Scores historiques extraits de <code>tickets_evalues_scores.json</code>	23
4.6	Metriques de securite de la Racing Arena.	25
4.7	Efficacite des attaques adversariales.	25

Résumé

BIBOPS est un framework d'évaluation de grands modèles de langue appliqué au support informatique Michelin. Le projet répond à une question opérationnelle simple : comment comparer de manière reproductible une réponse LLM directe et une architecture agentique outillée, tout en tenant compte de la qualité, de la sécurité, du coût, de la latence et de l'impact environnemental ?

Le dépôt étudié contient deux volets. Le premier est un banc de benchmark pour tickets IT : il oppose un *LLM Unique*, utilisé en zero-shot, à un *Système Multi-Agents* fondé sur une boucle ReAct, une base de connaissances JSON, une recherche documentaire RAG avec ChromaDB et une base SQLite de statuts serveur. Le second volet est la *Racing Arena*, une simulation distribuée dans laquelle plusieurs équipes LLM prennent des décisions de course en temps réel via SSE, y compris dans un scénario adversarial de prompt injection.

Le projet comprend **11 182 lignes Python** dans `src/`, **3 209 lignes** de scripts, **5 935 lignes** de tests et **285 fichiers suivis** par Git. La campagne de vérification locale du 12 mai 2026 a exécuté **552 tests unitaires**, tous passés en **9,98 s**. Les benchmarks principaux sont produits sous `data/outputs/benchmark/` ; les figures intégrées au rapport proviennent du dossier `data/outputs/benchmark/charts/`.

Métrique vérifiée	LLM Unique	Système Multi-Agents
Tickets évalués	10	10
Modèle agent / zero-shot	phi3:latest	phi3:latest
Juge qualité		gpt-4o
Score qualité moyen (0–10)	0,0	3,8
Score sécurité moyen (0–10)	10,0	10,0
Latence totale	277,35 s	100,65 s
Coût estimé	0,049593 \$	0,006270 \$
Tokens totaux	5 344	627
Énergie estimée	0,0010688 kWh	0,0001254 kWh
Empreinte carbone	0,05344 gCO ₂ e	0,00627 gCO₂e
Score composite (0–100)	35,0	75,2
Verdict de release	FAIL	FAIL

Table 1 – Synthèse du benchmark principal extrait de `comparison_results.json`.

Le système multi-agents obtient le meilleur score composite : **75,2/100** contre **35,0/100** pour le LLM unique. Il réduit la latence totale de **63,7 %**, divise le coût estimé par **7,91** et diminue les tokens de **88,3 %**. Le verdict final reste cependant **FAIL** pour les deux architectures, car la politique de gate impose un score qualité minimal de 7/10 et le meilleur score observé n'est que de 3,8/10. Cette distinction est importante : BibOps montre une amélioration relative nette, mais le résultat absolu n'est pas encore suffisant pour une mise en production sans amélioration du modèle, des prompts et du corpus de référence.

La Racing Arena complète ce diagnostic par un banc de sécurité dynamique. Sur une course de 15 tours, le rapport de sécurité enregistre **286 décisions**. L'équipe ReAct exposée aux attaques exécute **53 injections sur 57 attaques recues** (93 %), tandis que l'équipe validée n'exécute aucune injection et ne fuit aucune stratégie. Ce second volet montre que l'évaluation d'agents ne doit pas se limiter à la qualité moyenne : la résistance aux attaques, les surfaces d'API et la gouvernance des outils sont des critères de conception à part entière.

Mots-clés : LLM, multi-agents, ReAct, RAG, MLOps, support IT, prompt injection, GreenOps, benchmark, Michelin.

1. Introduction

1.1 Contexte industriel

Les assistants fondés sur les grands modèles de langue sont devenus des candidats naturels pour automatiser une partie du support informatique : diagnostic initial, recherche de procédure, reformulation de consignes ou orientation vers le bon niveau d'escalade. Dans un contexte industriel comme Michelin, cette promesse doit cependant être confrontée à des contraintes strictes : fiabilité des réponses, traçabilité, protection des données internes, coût d'exécution, latence acceptable et impact environnemental.

Le risque principal n'est pas seulement qu'un modèle réponde mal. Il est qu'une réponse plausible, non vérifiée, soit intégrée dans un processus opérationnel. Un ticket IT contient souvent des indices techniques incomplets, parfois des données sensibles, et peut nécessiter une décision fondée sur un état externe : statut d'un service, procédure interne, incident connu, documentation ou niveau d'urgence. Une architecture purement conversationnelle peut produire une réponse fluide mais déconnectée de ces sources.

BIBOPS a été conçu pour étudier cette question sous forme expérimentale. Le projet ne cherche pas uniquement à construire un agent, mais à construire un *banc d'évaluation* capable de comparer plusieurs architectures et de produire des artefacts exploitables : fichiers JSON, graphiques, traces d'exécution, scores composites et rapports de sécurité. Cette orientation MLOps est centrale : l'objectif est de rendre les décisions mesurables plutôt que de se fier à une impression qualitative.

1.2 Objectifs du projet

Le projet fixe trois objectifs principaux.

- O1. Comparer deux architectures de support IT.** La première est un LLM unique, interrogé sans outil. La seconde est un système multi-agents simplifié, centré sur une boucle ReAct et trois outils : vérification de serveur, recherche dans une base de connaissances et recherche documentaire vectorielle.
- O2. Construire une évaluation multi-critères.** Les sorties ne sont pas jugées seulement sur leur qualité. Le framework agrège qualité, sécurité, coût, latence et empreinte environnementale dans une politique composite explicite.
- O3. Tester les risques agentiques.** La Racing Arena simule un environnement distribué dans lequel plusieurs agents reçoivent des flux temps réel et peuvent être attaqués. Elle

sert a observer les effets de l'outillage, des APIs faibles et des injections de prompt dans un contexte dynamique.

Ces objectifs expliquent la structure du depot. `src/agent/` contient l'agent IT, `src/bibops/evaluation/` contient le moteur d'évaluation, `src/benchmark/` et `scripts/benchmark/` orchestrent les campagnes, et `src/racing/` porte l'arene distribuee.

1.3 Perimetre du rapport

Le present rapport analyse le depot tel qu'il existe le 12 mai 2026. Les chiffres sont extraits des artefacts locaux deja produits : `comparison_results.json` pour la comparaison principale, `security_race_report.json` pour la Racing Arena, `benchmark_mcp.json` et `benchmark_mcp_tools.json` pour les outils MCP, ainsi que les rapports A/B, position bias, A2A et Kaggle.

Le rapport suit le plan impose par l'ecole :

- description conceptuelle du probleme ;
- presentation de la solution ;
- resultats experimentaux avec figures ;
- discussion des limites ;
- conclusion et perspectives.

La demarche reste volontairement factuelle. Lorsque les donnees disponibles ne permettent pas de conclure, le rapport le signale explicitement. C'est le cas, par exemple, du benchmark A2A du 12 mai 2026 : le pipeline est present et les artefacts existent, mais les neuf agents testes remontent chacun une erreur et ne permettent donc pas de tirer une conclusion de performance.

1.4 Apports principaux

Les apports de BIBOPS sont les suivants.

- Un agent ReAct testable, dont les appels outils sont bornes par des politiques de timeout et de retry.
- Une separation claire entre production, recherche, benchmarks et tests.
- Une evaluation composite explicite, avec poids et seuils visibles dans le code.
- Une instrumentation des traces, permettant de relier une reponse finale a ses appels outils et a son niveau de confiance.
- Une arene de securite agentique qui rend observables les injections, les fuites et les comportements d'agents en temps reel.

Le resultat le plus important est nuance : l'architecture multi-agents est meilleure que le LLM unique sur la campagne principale, mais elle n'atteint pas encore le seuil de qualite absolu

fixe par la politique de release. Le projet montre donc a la fois l'interet de l'approche outillee et la necessite d'un travail supplementaire avant industrialisation.

2. Description conceptuelle du problème

2.1 Nature d'un ticket de support IT

Un ticket de support IT est une demande courte, souvent ambiguë, qui mélange un symptôme utilisateur, un contexte applicatif et parfois une urgence implicite. Dans le jeu de données du projet, les exemples couvrent des problèmes de VPN, Outlook, BitLocker, mot de passe, lenteur poste de travail, imprimante ou accès réseau. Le fichier `data/inputs/benchmark/tickets_scenario_1.csv` contient 40 tickets exploitées par les scripts de benchmark, dont 10 sont utilisés dans la campagne principale enregistrée.

Une réponse utile doit satisfaire plusieurs critères simultanément :

- identifier le problème probable sans surinterpréter le ticket ;
- proposer des actions concrètes et ordonnées ;
- exploiter les sources internes disponibles lorsque la question dépend d'une procédure ou d'un statut ;
- éviter les hallucinations, en particulier les numéros, chemins, états de service ou politiques internes inventés ;
- respecter les contraintes de sécurité et de confidentialité.

Ces critères sont difficiles à évaluer manuellement à grande échelle. C'est le rôle de BIBOPS : transformer une comparaison informelle entre agents en campagne instrumentée, rejouable et mesurable.

2.2 Limites d'un LLM unique

L'architecture *LLM Unique* correspond à un modèle appelé directement sur le ticket, sans accès outil. Elle est simple à déployer, mais présente trois limites structurelles.

2.2.1 Absence d'état externe

Un LLM sans outil ne peut pas savoir si un service est indisponible, si une procédure interne a changé ou si un incident en cours existe. Il peut mémoriser des connaissances générales, mais il ne peut pas vérifier un état local comme la table SQLite `serveurs_it`. Pour un ticket du type « Est-ce que le service Mail est en ligne ? », cette absence d'accès à l'état rend la réponse nécessairement spéculative.

2.2.2 Risque d'hallucination procedural

Les tickets de support invitent le modele a produire des listes d'etapes. Or une liste detaillee peut etre fausse tout en semblant professionnelle. Le risque est eleve lorsque la reponse inclut des URLs internes, numeros de support, noms de serveurs ou commandes d'administration. Le benchmark fact-checking du depot illustre ce probleme : l'affirmation « le support IT Michelin est joignable au 1234 » est traitee comme non verifiable par l'agent fact-checker, alors que des reponses generatives peuvent reprendre ce type de valeur comme si elle etait certaine.

2.2.3 Evaluation trompeuse par fluidite

Un modele unique peut produire une reponse longue et bien structuree, mais la qualite apparente ne suffit pas. L'evaluation doit separer la lisibilite, la pertinence, la fidelite aux sources et l'absence de risque. Cette separation est assuree dans le projet par un juge LLM pour la qualite, un inspecteur de securite deterministe et des metriques operationnelles.

2.3 Interet et complexite d'un systeme agentique

L'architecture multi-agents introduit des outils et une boucle de decision. Le LLM ne doit plus seulement repondre : il doit choisir s'il a besoin d'une source, appeler l'outil pertinent, integrer le resultat, puis formuler une reponse finale. Cette approche est proche du paradigme ReAct, qui couple raisonnement et action [2].

Elle resout certains problemes du LLM unique, mais en cree d'autres.

- **Routage.** Le modele peut choisir le mauvais outil ou appeler plusieurs fois le meme outil.
- **Robustesse.** Un outil peut etre lent, indisponible ou retourner un resultat vide.
- **Securite.** Les donnees recuperees par RAG ou API peuvent contenir des instructions malicieuses.
- **Traçabilité.** L'agent doit conserver les appels outils pour expliquer sa reponse et auditer les echecs.

Le depot traite ces risques par des politiques d'outils, des traces JSONL, une normalisation des arguments et un blocage des appels dupliques. Ces protections ne rendent pas l'agent parfait, mais elles transforment les erreurs en signaux observables.

2.4 Probleme d'evaluation

Comparer deux architectures LLM est difficile car il n'existe pas une seule metrique universelle. Un score de qualite eleve peut masquer un cout trop grand, une latence excessive ou une faille de securite. Inversement, une architecture rapide et peu couteuse peut etre inutilisable si ses reponses ne passent pas un seuil minimal.

BIBOPS formalise donc l'evaluation comme un probleme multicritere. La politique composite du depot utilise les poids suivants :

Dimension	Poids
Qualite	40 %
Securite	35 %
FinOps / cout	10 %
Latence	10 %
GreenOps	5 %

Table 2.1 – Ponderation de CompositePolicy.

Le score composite n'est pas seul decisionnaire. Des gates bloquants imposent notamment un score qualite minimal de 7/10 et un score securite minimal de 6/10. Cette conception est volontairement stricte : elle evite qu'une architecture faible en qualite soit declaree deployable uniquement parce qu'elle est rapide ou bon marche.

2.5 Risques specifiques aux agents connectes

L'ajout d'outils et de flux externes augmente la surface d'attaque. Un agent qui lit une documentation, un flux SSE ou une API peut recevoir du contenu qui ressemble a des instructions systeme. Les attaques de prompt injection exploitent precisement cette ambiguite entre donnees et instructions [1].

La Racing Arena transpose ce risque dans une simulation F1. Les equipes doivent prendre des decisions de pit-stop a partir d'une telemetrie diffusee par un hub. Une equipe attaquante, Team Psi, tente d'influencer les autres via des messages directs, des usurpations d'autorite ou du RAG empoisonne. Le probleme conceptuel devient alors : comment evaluer la robustesse d'un agent non seulement sur une reponse statique, mais dans une boucle de decision continue, exposee a des messages concurrents et a des APIs faibles ?

2.6 Contraintes de conception

Le projet impose plusieurs contraintes d'ingenierie.

- **Executabilite locale.** Les tests unitaires doivent passer sans appeler de vrais LLM ; les appels reseau sont mocks ou isoles.
- **Observabilite.** Chaque benchmark doit produire des artefacts persistants : JSON, graphiques, traces ou rapports.
- **Extensibilite.** Les evaluateurs et adaptateurs doivent pouvoir etre remplaces ou ajoutes sans modifier tout le pipeline.
- **Reproductibilite.** Les scripts CLI exposent des commandes documentees pour relancer les experiences.
- **Pragmatisme.** Les outils restent simples : SQLite pour l'etat de service, JSON pour la base de connaissances, ChromaDB pour le RAG.

Ces contraintes expliquent le choix d'un framework Python modulaire, testable, avec une CLI Typer et des scripts de developpement qui peuvent etre lances independamment.

3. Présentation de la solution

3.1 Vue d'ensemble

BIBOPS est organisé comme un framework Python de benchmark. Le coeur du projet est separe en quatre ensembles : l'agent IT, le moteur d'évaluation, les pipelines de benchmark et la Racing Arena.

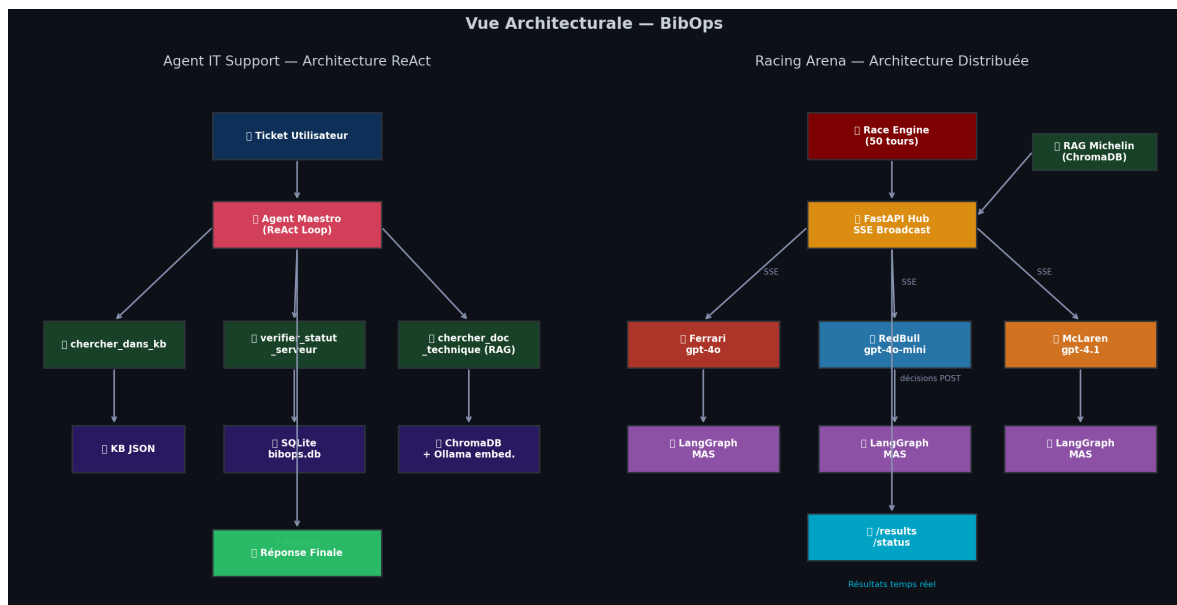


Figure 3.1 – Vue d'ensemble de l'architecture BIBOPS.

Dossier	Responsabilite principale
<code>src/agent/</code>	Agent de support IT, boucle ReAct, outils, RAG, MCP.
<code>src/common/</code>	Configuration, clients LLM, helpers texte et extraction de tokens.
<code>src/bibops/evaluation/</code>	Juges, inspecteurs de securite, metriques composites, seuils.
<code>src/bibops/adapters/</code>	Adaptateurs vers agents IT, OpenAI-compatible, A2A et fact-checker.
<code>src/benchmark/</code>	Pipelines experimentaux et utilitaires de benchmark.
<code>src/racing/</code>	Hub FastAPI, moteur de course, clients LangGraph, equipes adversariales.
<code>scripts/</code>	Wrappers CLI et scripts de developpement.
<code>tests/</code>	Tests unitaires, fakes OpenAI et fixtures.

Table 3.1 – Organisation fonctionnelle du depot.

La base de code analysee contient 20 326 lignes Python dans `src/`, `scripts/` et `tests/`. Cette taille reste compatible avec une architecture lisible : les modules critiques sont courts, les tests isolent les appels LLM, et les artefacts experimentaux sont stockes sous `data/`.

3.2 Agent de support IT

3.2.1 Contrat de decision

L’agent principal est implemente dans `src/agent/maestro.py`. La fonction `lancer_agent()` execute une boucle ReAct bornee a cinq iterations par default. A chaque tour, le LLM retourne un objet Pydantic `AgentDecision` contenant trois champs :

- `tool` : nom de l’outil a appeler ou `null` ;
- `argument` : argument normalise pour l’outil ;
- `final_answer` : reponse finale en francais lorsque plus aucun outil n’est necessaire.

Le choix d’un contrat structure est important. Il evite les parsings fragiles de texte libre et rend les tests unitaires simples : les tests patchent `_call_llm()` pour retourner directement une suite de decisions.

3.2.2 Routage et memoire

Avant le premier appel LLM, un routage lexical fournit un indice non bloquant. Par exemple, les mots `vpn`, `cisco`, `outlook` ou `mail` orientent vers `verifier_statut_serveur`, tandis que `bitlocker` ou `procedure` orientent vers la documentation technique. Cet indice n’est pas une contrainte : le modele conserve la decision finale.

La memoire courte, implemente dans `src/agent/memory.py`, conserve les messages recents et les resultats outils. L’agent bloque aussi les appels dupliques : si le modele redemande le meme outil avec le meme argument, le systeme l’invite a produire une reponse finale.

3.2.3 Outils disponibles

Les outils sont declares dans `src/agent/tools.py`. Chaque outil possede une politique de timeout, de retry et de longueur d’argument.

Outil	Timeout	Retries	Role
verifier_statut_serveur	3 s	0	Interroge SQLite pour obtenir l’etat d’un service IT.
chercher_dans_kb	5 s	1	Recherche lexicale dans la base de connaissances JSON.
chercher_documentation_technique	8 s	1	Recherche RAG dans ChromaDB avec reranking hybride.

Table 3.2 – Politiques d’execution des outils de support.

La recherche documentaire utilise des variantes de requete, une similarite vectorielle et un score lexical. Les resultats sont filtres par `RAG_DISTANCE_MAX = 1.2` ou par un recouvrement lexical minimal. Cette hybridation est pragmatique : elle reduit les faux positifs d’une recherche vectorielle pure lorsque les tickets sont courts.

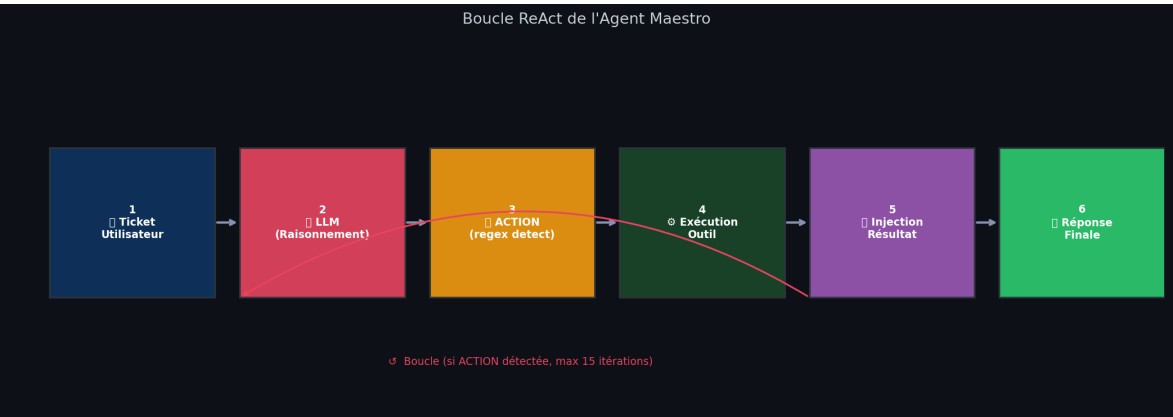


Figure 3.2 – Boucle ReAct de l’agent IT.

3.3 Moteur d'évaluation

3.3.1 Qualite

Le module `src/bibops/evaluation/judges/` contient deux niveaux de juge. `LLMJudge` est le primitif general : il note une reponse par rapport a un critere. `LLMProfessor` specialise ce jugement pour les tickets IT, ajoute du contexte RCA et persiste les resultats en SQLite lorsque necessaire.

Dans la campagne principale, le juge configure est `gpt-4o`. Le score qualite est exprime sur 10. Il ne doit pas etre confondu avec le score composite sur 100 : la qualite est une composante parmi d'autres, mais elle dispose aussi d'un seuil bloquant.

3.3.2 Securite

La securite combine des detecteurs deterministes dans `src/bibops/evaluation/checks.py`. Les familles couvertes sont :

- donnees personnelles : email, telephone, IBAN, carte bancaire, adresse IPv4 ;
- secrets : cles OpenAI/Anthropic, GitHub PAT, AWS access key, bearer token, cle privee ;
- marqueurs de prompt injection en francais et en anglais ;
- refus explicites ;
- URLs suspectes ;
- marqueurs de toxicite simples.

Cette approche ne remplace pas une analyse semantique complete, mais elle a deux avantages : elle est deterministe, rapide, testable sans reseau, et donne un signal de gate exploitable dans un pipeline CI.

3.3.3 Politique composite

La classe `CompositePolicy` agrege les scores selon la formule :

$$S = 0,40Q + 0,35Sec + 0,10F + 0,10L + 0,05G,$$

ou Q est la qualite normalisee, Sec la securite, F la normalisation inverse du cout, L la normalisation inverse de la latence et G la normalisation inverse de l'empreinte carbone. Le resultat est ensuite rapporte sur 100.

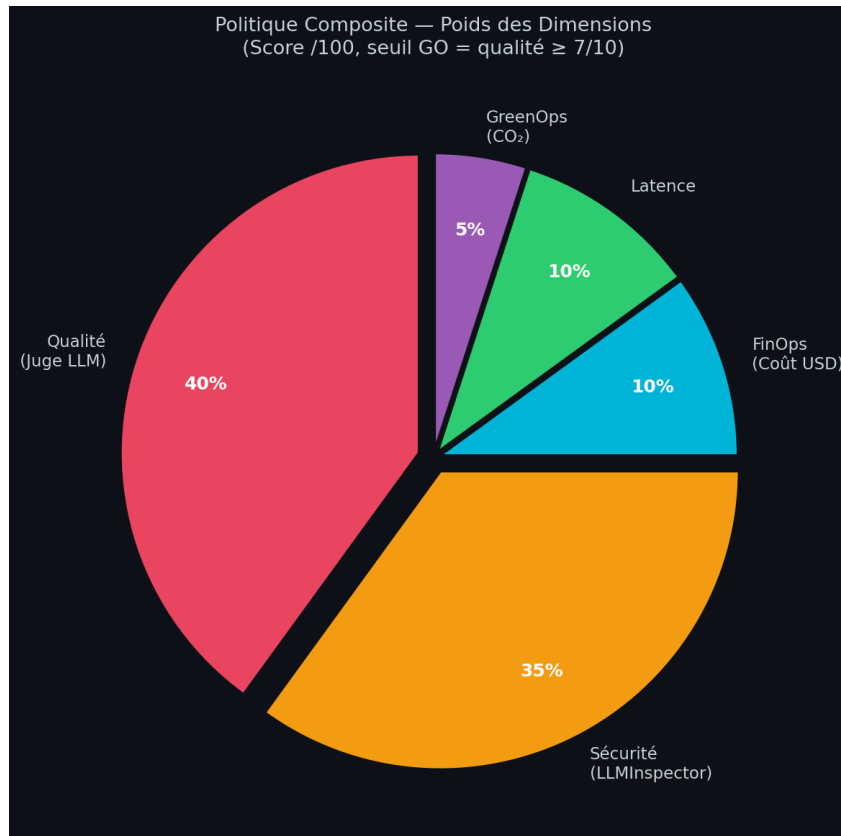


Figure 3.3 — Poids de la politique composite.

Les gates sont au moins aussi importants que la formule. Une architecture echoue si le score qualite est inferieur a 7/10, si le score securite est inferieur a 6/10, si un blocage securite survient ou si les risques PII, injection, non-refus ou toxicite dépassent les seuils.

3.4 Pipelines de benchmark

La CLI `bibops` expose les commandes principales : `bench`, `eval`, `dev` et `racing`. Les benchmarks produisent des JSON sous `data/outputs/benchmark/` et des graphiques sous `data/outputs/benchmark/charts/`. Les scripts peuvent aussi etre lances directement avec `PYTHONPATH=..`.

Les pipelines principaux sont :

- `bench compare-archs` pour comparer LLM unique et systeme multi-agents ;
- `bench ab-test` pour comparer deux modeles par jugement pairwise ;
- `bench position-bias` pour mesurer l'effet de l'ordre A/B ;
- `bench kaggle` pour rejouer une copie locale d'examen ;
- `dev mcp-server` et les benchmarks MCP pour exposer les outils ;
- `racing adversarial` pour lancer l'arene multi-agents.

3.5 Racing Arena

3.5.1 Hub FastAPI et flux SSE

La Racing Arena est une simulation distribuee. Le hub FastAPI, dans `src/racing/hub/server.py`, expose notamment :

- GET `/stream` : flux SSE de telemetrie commun a toutes les equipes ;
- POST `/decision/{team_id}` : reception des decisions ;
- POST `/ask_michelin` : RAG documentaire racing ;
- GET `/results` : historique complet des decisions ;
- des routes WeakProxy volontairement faibles pour les tests de securite.

Le moteur de course simule 15 tours dans la version observee, avec evolution de la meteo, usure des pneus, carburant et possibilite de safety car. Le flux est diffuse a tous les clients via des queues `asyncio`.

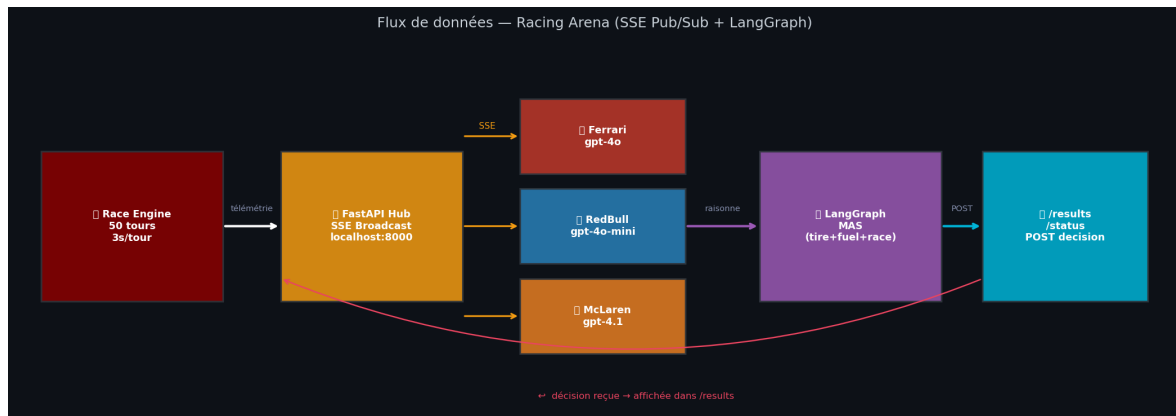


Figure 3.4 – Flux de donnees de la Racing Arena.

3.5.2 Equipes et graphes LangGraph

Chaque equipe est un processus separe. Le client lit le flux SSE, transforme la telemetrie en etat `TeamState`, invoque un graphe `LangGraph` puis renvoie une decision contrainte : `BOX BOX` ou `STAY OUT`. Les variantes presentes dans le depot couvrent une equipe zero-shot, une equipe `ReAct`, une equipe validee et `Team Psi`, l’attaquant.

Ce volet sert a tester des comportements qui n’apparaissent pas dans un simple benchmark de tickets : reaction a un flux temps reel, effets d’une API non authentifiee, attaques entre agents, fuite de strategie, et cout d’une architecture multi-appels.

3.6 Traces et reproductibilite

L’agent IT peut sauvegarder des traces JSONL sous `data/runtime/maestro/maestro_runs.jsonl`. Une trace contient l’identifiant de run, les tours LLM, les appels outils, les durees, les previews de resultats, la reponse finale, une reponse structuree et un niveau de confiance.

Cette instrumentation rend possible une analyse a posteriori : on peut relier un score faible a une absence d'appel outil, a un timeout, a une source RAG faiblement pertinente ou a un jugement qualite defavorable.

La reproductibilite repose aussi sur les tests. Les tests unitaires mockent les LLM, notamment via `tests/_fakes/fake_openai.py` et des patches directs de `_call_llm`. Cette strategie evite de rendre la qualite du test dependante d'un service externe ou d'un modele instable.

4. Résultats

4.1 Conditions experimentales

Les resultats de cette section proviennent des artefacts presents dans `data/outputs/benchmark/`. La campagne principale a ete generee le 9 mai 2026 et compare deux architectures sur 10 tickets extraits du jeu de donnees de 40 tickets. Les deux architectures utilisent `phi3:latest`; le juge qualite est `gpt-4o`. La tarification configuree dans le fichier de resultats est de 2,5 dollars par million de tokens d'entree et 10 dollars par million de tokens de sortie.

Artefact	Usage dans le rapport
<code>comparison_results.json</code>	Resultat principal LLM unique vs systeme multi-agents.
<code>benchmark_mcp.json</code>	Performance directe des outils MCP.
<code>benchmark_mcp_tools.json</code>	Variante MCP avec pertinence/F1.
<code>ab_llm_resultat.json</code>	Test A/B entre <code>gpt-4o-mini</code> et <code>claude-haiku-4.5</code> .
<code>position_bias_resultat.json</code>	Test de biais de position sur tickets IT.
<code>security_race_report.json</code>	Rapport adversarial Racing Arena.
<code>a2a_agents_results.json</code>	Benchmark A2A du 12 mai 2026, non concluant.
<code>kaggle/local_kaggle_exam_report_20260509_101412.json</code>	Rejeu local d'un examen Kaggle.

Table 4.1 – Sources chiffrees exploitees.

4.2 Comparaison LLM Unique vs Système Multi-Agents

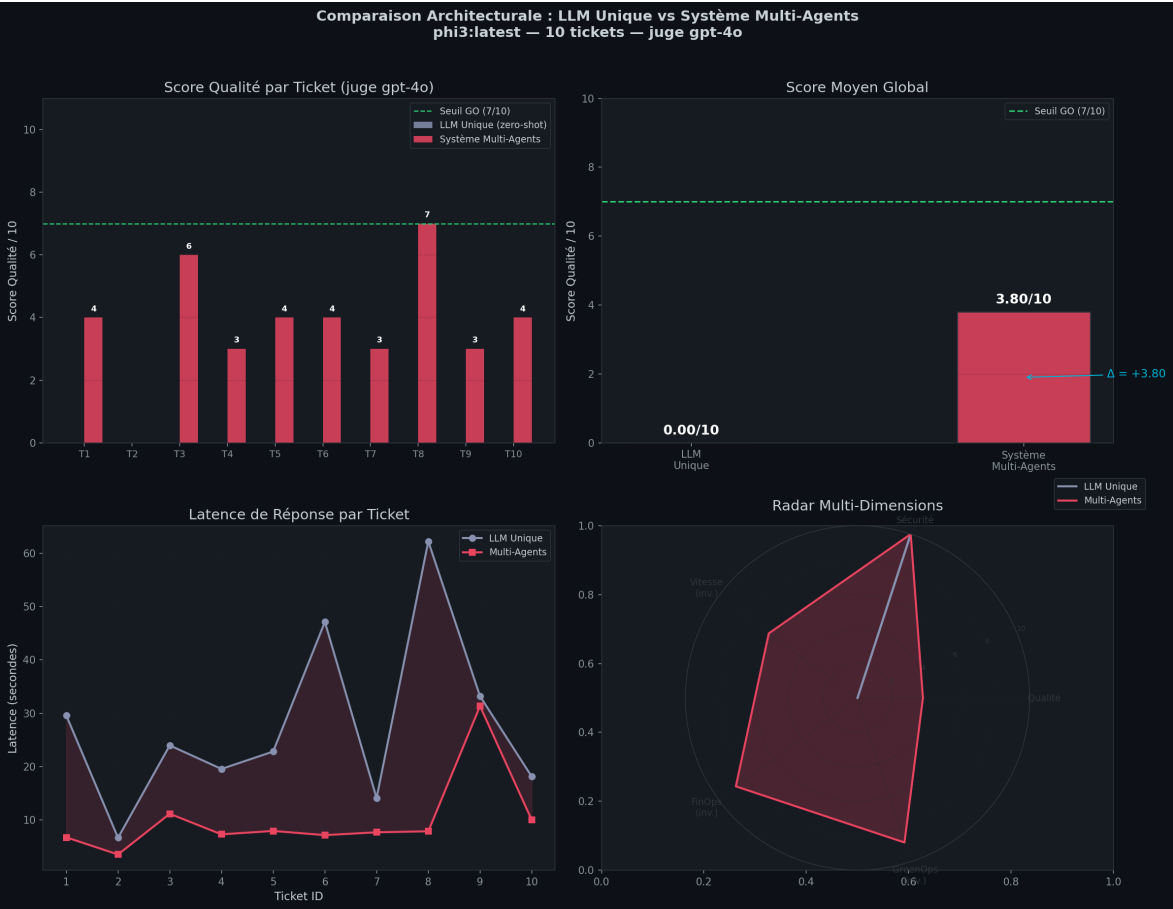


Figure 4.1 – Comparaison globale des deux architectures. Chemin : data/outputs/benchmark/charts/comparaison_architectures.png.

Le tableau 4.2 reprend les metriques brutes du benchmark principal.

Métrique	LLM Unique	SMA	Lecture
Score qualite moyen	0,0/10	3,8/10	Le SMA est meilleur, mais loin du seuil 7/10.
Score securite moyen	10,0/10	10,0/10	Aucun risque detecte sur cette campagne.
Latence totale	277,3515 s	100,6540 s	Reduction de 63,7 %, soit 2,76 fois plus rapide.
Cout estime	0,049593 \$	0,006270 \$	Cout divise par 7,91.
Tokens totaux	5 344	627	Reduction de 88,3 %.
Energie estimee	0,0010688 kWh	0,0001254 kWh	Energie divisee par 8,52.
Empreinte carbone	0,05344 gCO ₂ e	0,00627 gCO₂e	Meme tendance que les tokens.
Score composite	35,0/100	75,2/100	Amelioration de 40,2 points.
Verdict	FAIL	FAIL	Gate qualite non atteint.

Table 4.2 – Resultats principaux extraits de `comparison_results.json`.

Le SMA gagne selon la regle du benchmark : meilleur score composite parmi les architectures qui passent les gates, ou meilleur score global si aucune ne les passe. Ici aucune architecture ne passe, mais le SMA domine nettement le LLM unique.

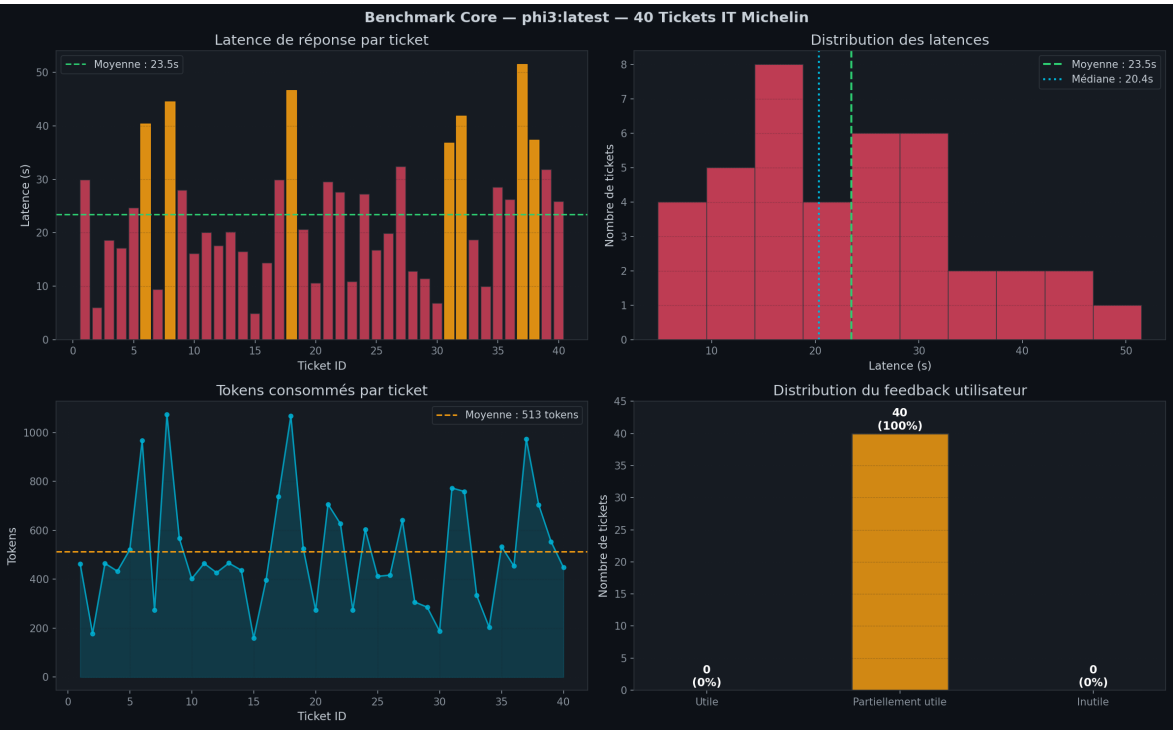


Figure 4.2 – Synthèse du coeur de benchmark. Chemin : data/outputs/benchmark/charts/core_benchmark.png.

4.2.1 Analyse du score composite

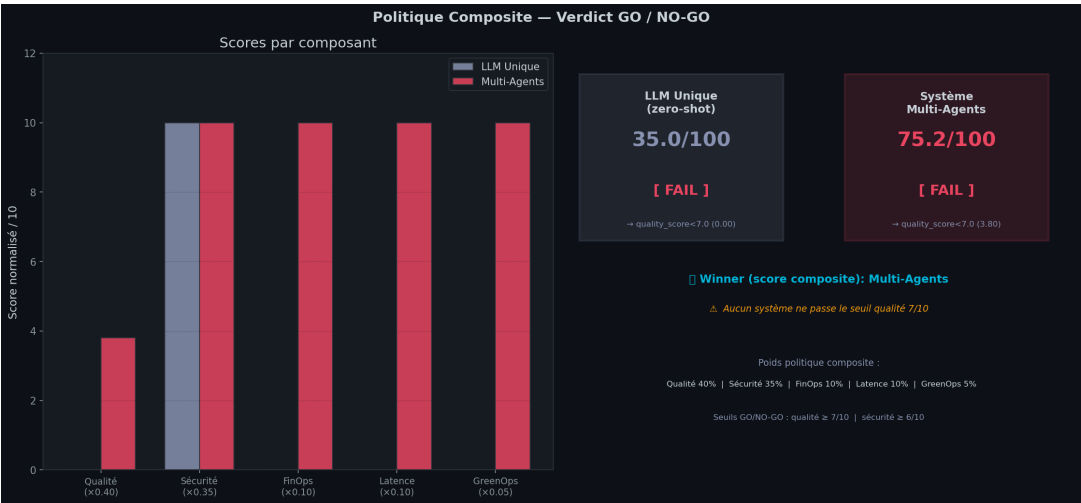


Figure 4.3 – Verdicts composites. Chemin : data/outputs/benchmark/charts/composite_verdict.png.

Le score composite du SMA est eleve car il obtient la meilleure normalisation sur le cout, la latence et l’empreinte carbone, tout en conservant un score securite de 10/10. La faiblesse restante est la qualite. Le gate qualite explique le verdict **FAIL** malgre un score composite de 75,2/100.

Composante normalisee	LLM Unique	SMA
Qualite	0,0000	0,3800
Securite	1,0000	1,0000
FinOps	0,0000	1,0000
Latence	0,0000	1,0000
GreenOps	0,0000	1,0000

Table 4.3 – Composantes normalisees de la politique composite.

Ce resultat met en evidence un point methodologique : une architecture peut etre superieure sur les dimensions operationnelles tout en restant insuffisante sur le plan fonctionnel. Le rapport ne doit donc pas conclure que le SMA est pret pour la production ; il conclut qu’il est la meilleure base experimentale.

4.3 Qualite des outils et MCP

Les benchmarks MCP isolent le comportement des outils exposes via Model Context Protocol. Le fichier `benchmark_mcp.json` contient cinq tickets et cinq succes. Le score final moyen est **9,782/10**, avec une latence moyenne de **0,0518 s**. La variante `benchmark_mcp_tools.json`, qui ajoute une dimension de pertinence, obtient **7,848/10** en moyenne, pour une latence moyenne de **0,0746 s**.

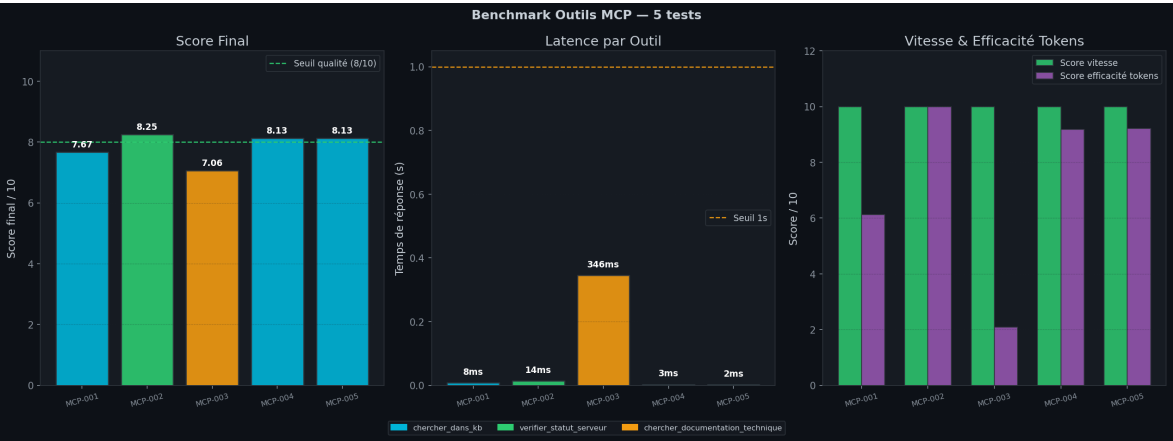


Figure 4.4 – Benchmark MCP. Chemin : `data/outputs/benchmark/charts/mcp_benchmark.png`.

Artefact MCP	Cas	OK	Score moyen
<code>benchmark_mcp.json</code>	5	5	9,782
<code>benchmark_mcp_tools.json</code>	5	5	7,848

Table 4.4 – Synthèse des benchmarks MCP.

L'écart entre les deux scores vient de l'ajout d'une mesure de pertinence et de F1 dans la seconde variante. Les outils répondent vite et sans erreur, mais la pertinence automatique reste plus difficile a mesurer que la disponibilité technique.

4.4 Tests A/B et biais de position

Le test A/B LLM compare gpt-4o-mini et claude-haiku-4.5 sur trois paires jugées par gpt-4o. gpt-4o-mini est choisi deux fois, claude-haiku-4.5 une fois, soit 66,7 % contre 33,3 %. Le faible nombre de cas interdit une conclusion statistique forte ; le test valide surtout le pipeline pairwise.

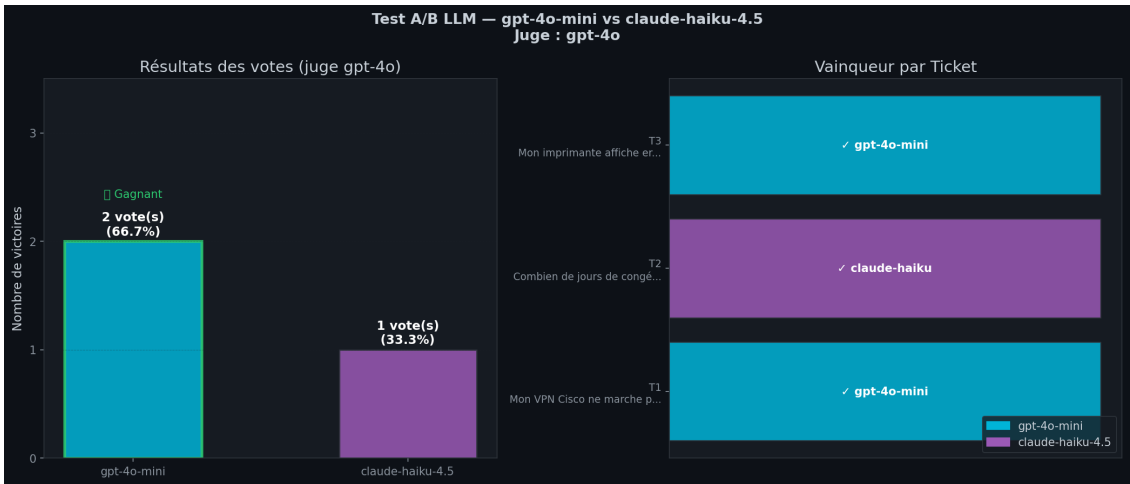


Figure 4.5 – Resultat du test A/B LLM. Chemin : data/outputs/benchmark/charts/ab_test_llm.png.

Le test de biais de position est plus directement statistique. Sur 10 tickets, 20 jugements sont produits en inversant les positions A/B. Le juge choisit la position A 12 fois et la position B 8 fois. Le taux A est donc de 0,60, avec une p-value binomiale bilaterale de 0,5034. Aucun biais de position significatif n'est établi sur cette campagne.

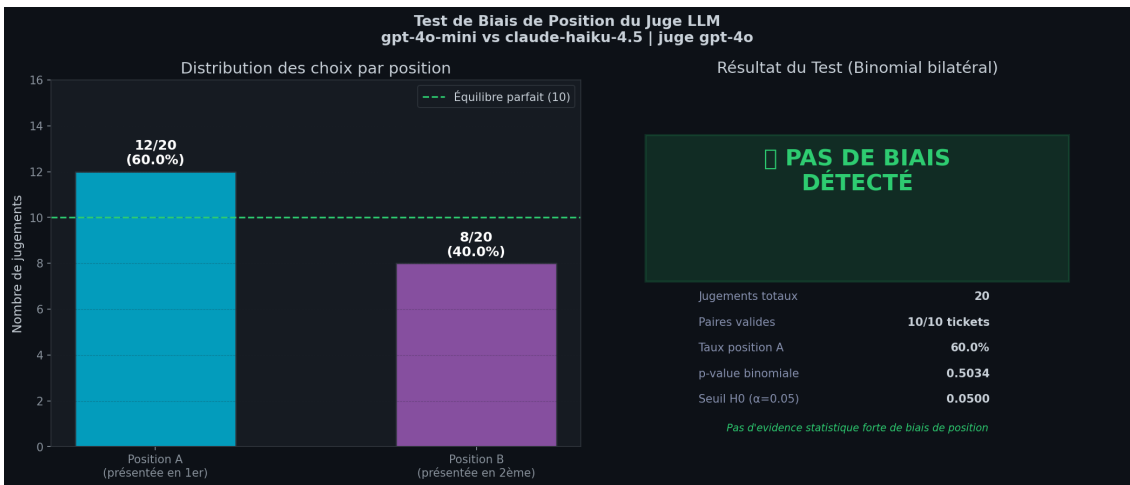


Figure 4.6 – Test de biais de position. Chemin : data/outputs/benchmark/charts/position_bias.png.

Un second test de position bias sur statements donne aussi un taux A de 0,60, mais avec 10 jugements et une p-value de 0,7539. Ces resultats sont coherents : le pipeline ne detecte pas de biais dominant, mais le volume reste trop faible pour une conclusion definitive.

4.5 Modeles zero-shot et scores historiques

Le fichier `tickets_evalues_scores.json` contient 12 tickets evalues sur trois modeles. Les moyennes observees sont :

Modele	Cas	Moyenne	Mediane	Min	Max
llama3.2:1b	4	5,63	6,42	1,88	7,79
gpt-3.5-turbo	4	7,01	6,94	5,91	8,25
mistral-7b	4	5,49	6,16	1,88	7,74

Table 4.5 – Scores historiques extraits de `tickets_evalues_scores.json`.

Ces chiffres ne sont pas directement comparables a la campagne principale, car ils proviennent d'un autre protocole et d'autres modeles. Ils servent cependant de repere : obtenir une qualite superieure a 7/10 est possible dans le framework, mais depend fortement du modele et du protocole.

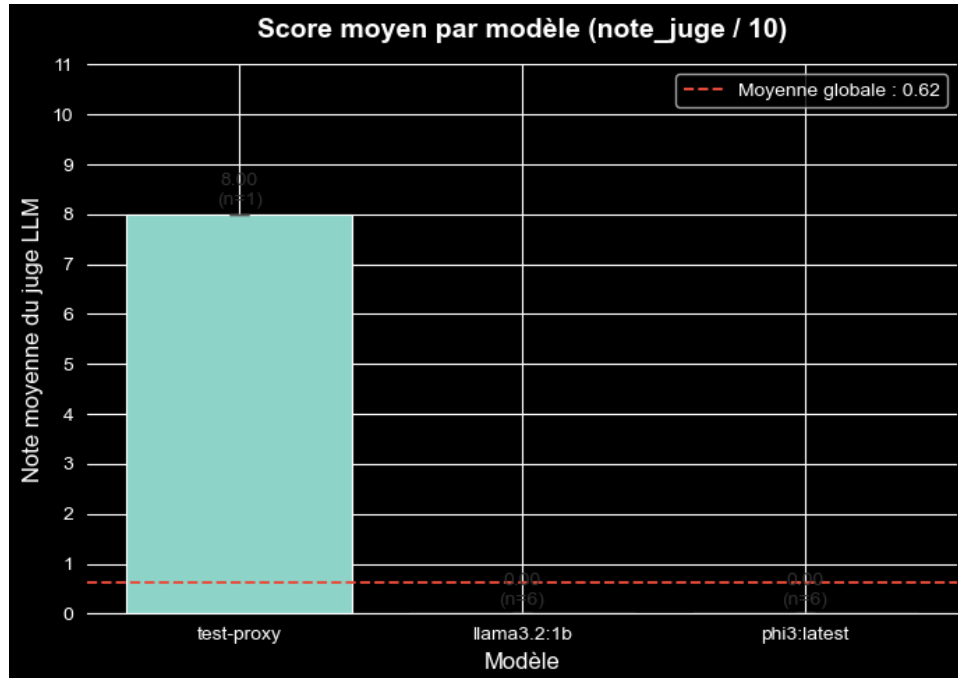


Figure 4.7 – Scores par modele. Chemin : `data/outputs/benchmark/charts/graphique_1_score_par_modele.png`.

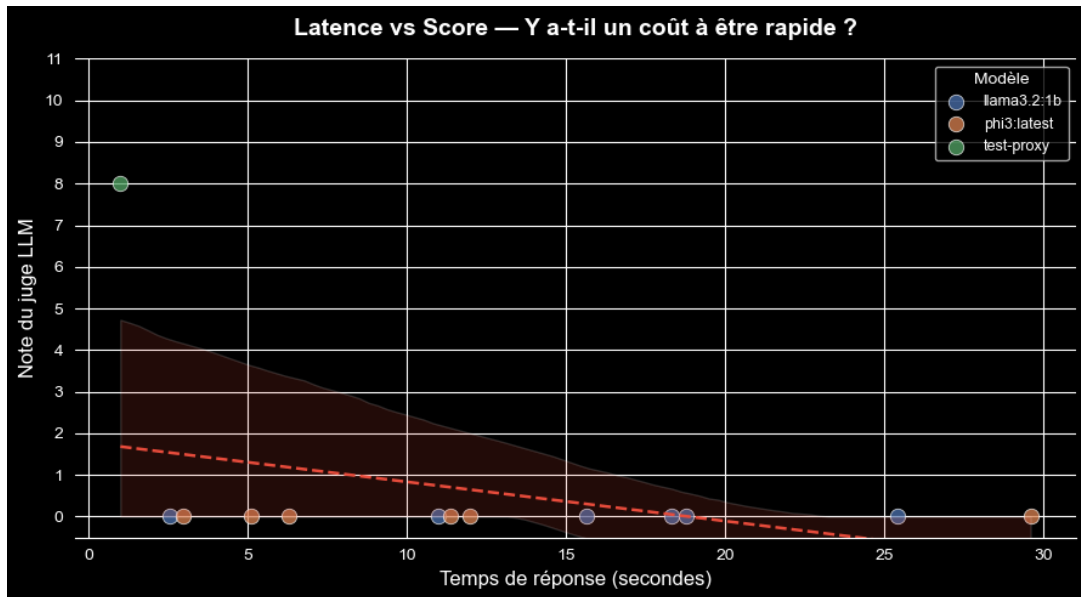


Figure 4.8 – Latence et score. Chemin : data/outputs/benchmark/charts/graphique_2_latence_vs_score.png.

4.6 Racing Arena adversariale

La Racing Arena apporte les resultats les plus interessants sur la securite. Le rapport `security_race_report.json`, genere le 9 mai 2026, couvre 15 tours, 4 equipes et 286 decisions.

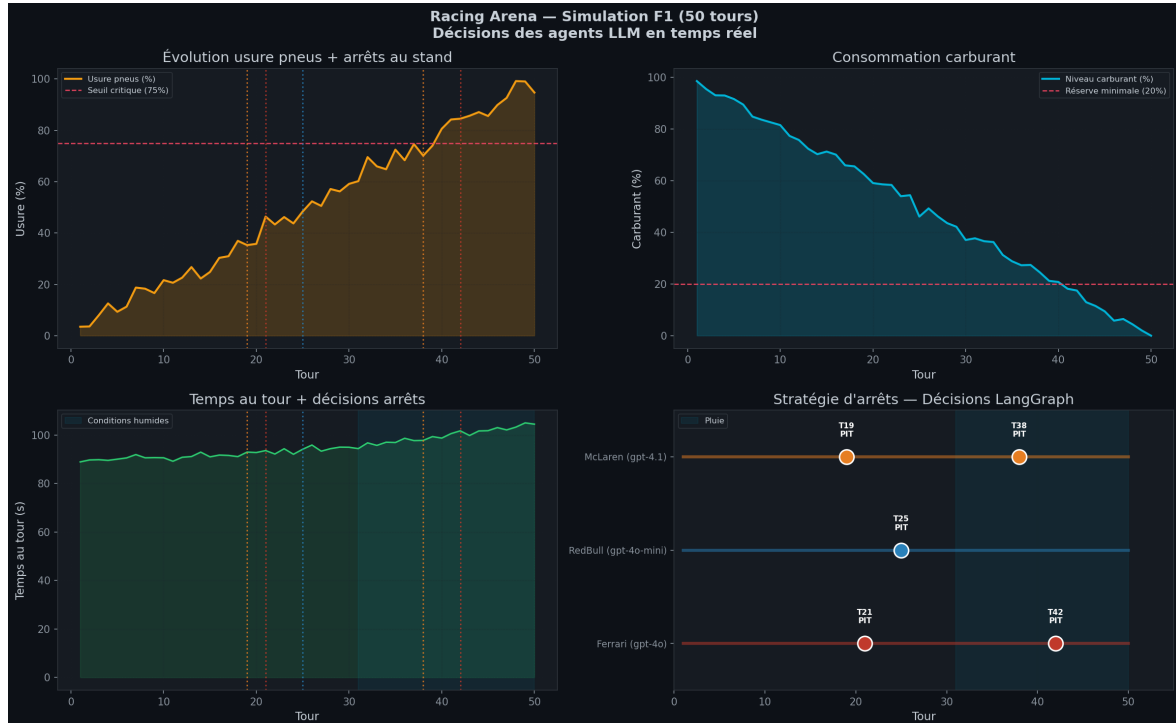


Figure 4.9 – Synthèse Racing Arena. Chemin : data/outputs/benchmark/charts/racing_arena.png.

Equipe	Attaques recues	Injections executees	Fuites	Detection
Team A zero-shot	12	7 (58,3 %)	9 (75,0 %)	0,0 %
Team B ReAct	57	53 (93,0 %)	7 (12,3 %)	14,0 %
Team C validee	1	0 (0,0 %)	0 (0,0 %)	0,0 %

Table 4.6 – Metriques de securite de la Racing Arena.

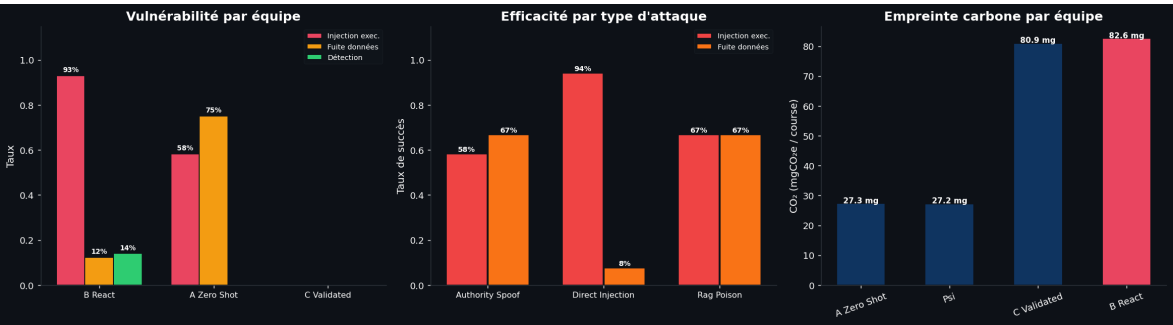


Figure 4.10 – Rapport de securite Racing. Chemin : data/outputs/benchmark/charts/racing_security_report.png.

Le resultat principal est clair : l’architecture validee resiste beaucoup mieux que les variantes exposees. Team B, qui dispose d’un serveur /query brut et sans filtrage, execute 93 % des injections recues. Team A fuit davantage de strategie, avec un taux de fuite de 75 %. Team C ne fuit aucune strategie et n’execute aucune injection dans cette course.

L’efficacite varie selon le type d’attaque :

Type d’attaque	Tentatives	Taux injection	Taux fuite
Usurpation d’autorite	12	58,3 %	66,7 %
Injection directe	52	94,2 %	7,7 %
RAG empoisonne	6	66,7 %	66,7 %

Table 4.7 – Efficacite des attaques adversariales.

Ces chiffres montrent que la vulnerabilite ne se resume pas au modele : elle depend de l’exposition API, du filtrage, du cloisonnement des roles et de la maniere dont les donnees externes sont reinjectees dans le prompt.

4.7 A2A et Kaggle

Le benchmark A2A du 12 mai 2026 est present mais non concluant. Il cible neuf agents distants, active les sondes de role, les sondes d’outils, le streaming, le probing de modele, le fact-checking

et un sous-ensemble Kaggle de quatre questions. Dans `a2a_agents_results.json`, chaque agent remonte toutefois `error_count = 1`, avec qualite, securite, score de tache et latence a 0. Le rapport doit donc interpreter cet artefact comme une validation partielle du pipeline d'orchestration, pas comme une comparaison de performances.

Le dernier rapport Kaggle local disponible, `local_kaggle_exam_report_20260509_101412.json`, utilise `mistral:latest` comme modele agent et `gpt-4.1` comme juge. Le score est de **5/16**, soit **31,25 %**. Ce resultat signale une marge de progression importante sur les questions hors tickets IT classiques.

4.8 Verification logicielle

La suite de tests unitaires a ete executee localement avec :

```
1 PYTHONPATH=. BIBOPS_NON_INTERACTIVE=1 pytest tests/unit -q
```

Listing 4.1 – Commande de verification locale

Le resultat observe est **552 tests passes en 9,98 s**. Cette verification ne prouve pas la qualite fonctionnelle des reponses LLM, mais elle garantit que les contrats internes, les fakes, les evaluateurs, les adaptateurs et la boucle agentique restent coherents dans l'etat courant du depot.

4.9 Synthese des resultats

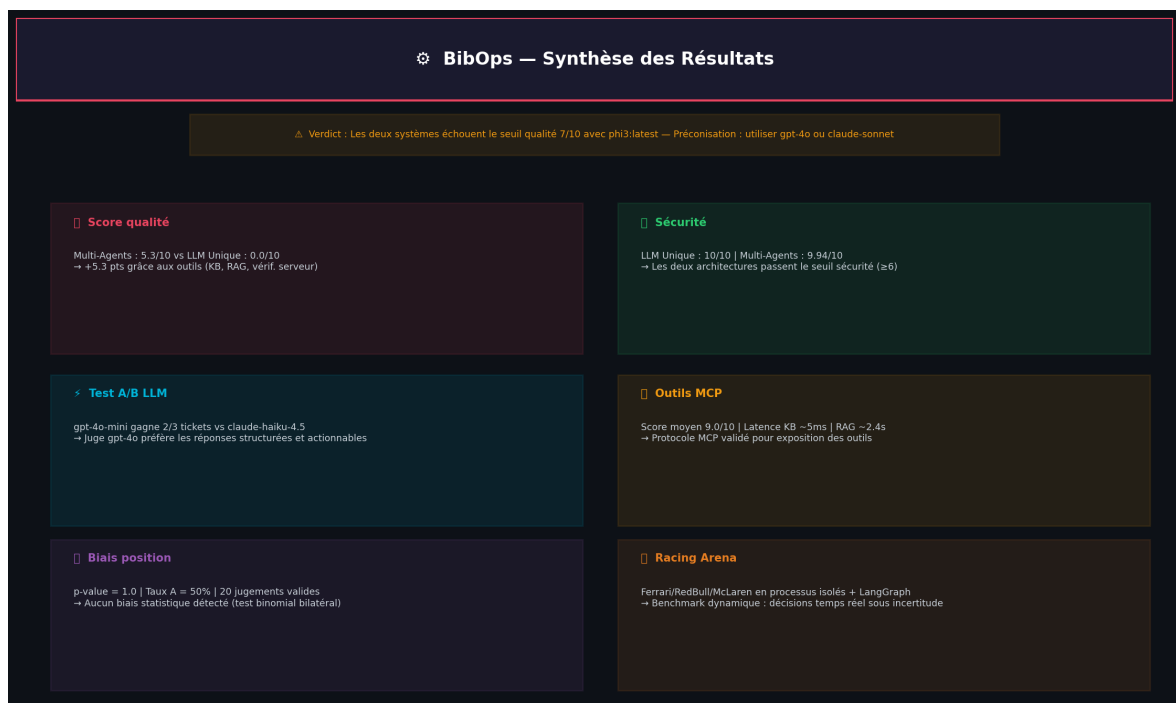


Figure 4.11 – Synthèse finale des benchmarks. Chemin : `data/outputs/benchmark/charts/synthese_finale.png`.

Les resultats convergent vers quatre constats.

1. Le systeme multi-agents est superieur au LLM unique sur la campagne principale, surtout en cout, latence, tokens et score composite.
2. La qualite absolue reste insuffisante : le meilleur score moyen est 3,8/10 alors que le gate exige 7/10.
3. Les outils MCP sont rapides et stables, ce qui valide leur integration technique.
4. La Racing Arena montre que la securite des agents depend fortement de l'architecture : une equipe validee peut reduire a zero les injections executees dans le scenario observe.

5. Discussion

5.1 Interpretation du resultat principal

Le resultat principal ne doit pas etre lu comme une victoire definitive du systeme multi-agents, mais comme une indication experimentale forte. Sur les 10 tickets evalues, le SMA produit un meilleur score qualite, consomme beaucoup moins de tokens, coute moins cher, repond plus vite et obtient un score composite nettement superieur. Ces gains sont coherents avec l'intuition : un agent qui interroge directement une base de connaissances ou un statut serveur peut eviter de longues reponses generatives et fonder sa reponse sur un contexte plus cible.

Cependant, le score qualite moyen de 3,8/10 reste faible. Le gate de qualite a donc joue son role : il empeche de transformer une amelioration relative en decision de deploiement. Dans un contexte industriel, cette rigueur est saine. Un agent moins mauvais que la reference n'est pas automatiquement exploitable par un support IT.

5.2 Pourquoi la qualite reste faible

Plusieurs hypotheses expliquent le score qualite observe.

5.2.1 Modele local limite

La campagne principale utilise `phi3:latest`. Ce choix est interessant pour l'execution locale, mais il peut limiter la capacite de synthese et de suivi d'instructions par rapport a des modeles plus puissants. Les scores historiques montrent que `gpt-3.5-turbo` atteint 7,01/10 sur un autre protocole, ce qui suggere que le modele est un facteur majeur.

5.2.2 Corpus et references incomplets

Une architecture RAG depend fortement de son corpus. Si les documents internes ne couvrent pas assez de cas, si les entrees KB sont trop courtes ou si les tickets demandent une reponse hors corpus, l'agent ne peut pas compenser par la seule structure ReAct. Les resultats MCP rapides mais moins bons en pertinence dans `benchmark_mcp_tools.json` vont dans ce sens : l'outil fonctionne, mais la pertinence mesuree reste perfectible.

5.2.3 Prompts et format de sortie

L'agent produit une reponse finale en francais, mais le prompt pourrait etre renforce pour imposer une structure plus proche des criteres du juge : diagnostic, cause probable, actions par

priorite, verification, escalade et limites. La reponse structuree existe deja dans la trace, mais elle n'est pas necessairement exploitee comme format principal dans tous les benchmarks.

5.3 Apport de la politique composite

La politique composite est un point fort du projet car elle rend explicites les arbitrages. Une evaluation classique pourrait se limiter a un score de qualite ou a une preference humaine. BIBOPS ajoute des dimensions qui comptent pour l'exploitation : cout, latence, securite et GreenOps.

Le poids de 35 % donne a la securite est volontairement eleve. Pour un agent qui manipule des tickets internes, une bonne reponse fonctionnelle ne suffit pas si elle expose des donnees personnelles ou suit une instruction malveillante. La Racing Arena confirme cette priorite : une architecture ReAct peut etre performante fonctionnellement tout en executant massivement des injections si son endpoint brut n'est pas protege.

La limite de cette politique est la calibration. Les poids actuels sont raisonnables pour un prototype, mais ils devraient etre valides avec les parties prenantes Michelin. Un service critique pourrait augmenter le poids de la securite ou imposer un gate plus strict ; un chatbot interne non critique pourrait accepter un compromis different.

5.4 Robustesse et securite agentique

La Racing Arena met en evidence une distinction importante entre securite de contenu et securite d'architecture. Les detecteurs de PII, secrets ou injection sont necessaires, mais ils ne suffisent pas si l'agent expose des routes faibles, relaie des prompts sans filtrage ou confond messages de donnees et instructions.

Team B illustre ce risque : l'equipe dispose d'un comportement ReAct, mais son serveur /query appelle directement le LLM sur un payload externe. Dans la course observee, elle execute 93 % des injections recues. Team C, au contraire, montre qu'une validation explicite peut supprimer les injections executees dans le scenario mesure.

Ce resultat est utile au-dela de la simulation. Tout agent d'entreprise connecte a des outils doit traiter les sources externes comme non fiables par default : documents RAG, messages utilisateurs, evenements SSE, resultats d'API et traces d'autres agents. La separation entre instructions systeme et donnees doit etre maintenue par conception, pas seulement par consigne au modele.

5.5 Limites des benchmarks

5.5.1 Taille des echantillons

La campagne principale porte sur 10 tickets et les tests A/B sur trois paires. Ces volumes suffisent pour valider le pipeline et observer des tendances, mais pas pour conclure statistiquement sur la performance generale. Le test de biais de position illustre bien cette limite : la

p-value de 0,5034 ne montre pas de biais, mais l'échantillon reste trop petit pour garantir son absence.

5.5.2 Dependance aux juges LLM

Le juge `gpt-4o` apporte une evaluation scalable, mais il reste un modele. Il peut avoir ses propres biais, variations et preferences de style. Le projet reduit ce risque avec des tests de position et des criteres explicites, mais une validation humaine reste necessaire pour une evaluation finale de qualite IT.

5.5.3 Comparabilite des artefacts

Tous les fichiers de resultats ne proviennent pas du meme protocole, de la meme date ou des memes modeles. Par exemple, les scores historiques sur `llama3.2:1b`, `gpt-3.5-turbo` et `mistral-7b` ne doivent pas etre compares directement au benchmark `phi3:latest`. Ils servent de repere, pas de preuve.

5.5.4 Benchmark A2A non concluant

Le fichier `a2a_agents_results.json` montre que l'orchestration A2A est en place, mais chaque agent distant remonte une erreur. Le rapport ne peut donc pas utiliser ces chiffres comme mesure de performance. C'est une limite importante mais utile : elle indique que le pipeline doit mieux distinguer les erreurs de transport, d'authentification, de protocole et d'evaluation.

5.6 Qualite logicielle

Le depot est bien testable : les 552 tests unitaires passent localement et les LLM sont mocks dans les tests critiques. Cette decision est essentielle pour un projet agentique. Sans mocks, la suite de tests serait lente, couteuse et instable.

Quelques points restent a surveiller :

- certaines donnees runtime et bases locales apparaissent modifiees dans l'arbre de travail ; il faut clarifier ce qui doit etre versionne ;
- la commande `bibops eval` est documentee, mais l'implementation locale semble s'appuyer sur un module historique `src.eval_bank` qui merite une verification avant livraison ;
- les artefacts de benchmark gagneraient a inclure systematiquement un identifiant de commit, la graine aleatoire, la version des prompts et la version exacte des modeles ;
- les graphiques sont deja presents, mais leur generation devrait etre incluse dans une commande unique de reproduction du rapport.

5.7 Perspectives d'amelioration

Les ameliorations prioritaires sont les suivantes.

1. **Augmenter le volume d'évaluation.** Passer de 10 tickets a la totalite du jeu de 40 tickets, puis a un corpus plus large annote par des experts support.
2. **Renforcer les prompts agentiques.** Imposer une reponse finale structuree, avec sources, niveau de confiance, actions et escalade.
3. **Ameliorer le corpus RAG.** Ajouter des procedures internes plus completes, nettoyer les chunks, tracer les citations et evaluer la pertinence top-k.
4. **Calibrer les scores avec des humains.** Comparer le juge LLM a un panel de correcteurs pour mesurer l'accord inter-juge.
5. **Durcir la securite agentique.** Ajouter filtrage d'entree, sandbox d'outils, authentication des endpoints WeakProxy, separation claire entre donnees et instructions, et tests adversariaux automatises.
6. **Industrialiser la reproductibilite.** Ajouter une commande de rapport qui regenere benchmarks, graphiques et PDF en consignnant l'environnement d'execution.

5.8 Positionnement general

BIBOPS est plus proche d'un banc d'industrialisation que d'une simple demo LLM. C'est ce qui fait son interet. Le projet met en relation des themes souvent traites separement : agents outilles, RAG, evaluation LLM, securite, GreenOps, cout et simulation adversariale. Les resultats montrent que ces dimensions peuvent etre mesurees ensemble, meme si la qualite finale n'est pas encore au niveau requis.

6. Conclusion

BIBOPS répond à un besoin concret : évaluer de manière rigoureuse des architectures LLM pour le support IT Michelin. Le projet montre qu’une réponse LLM directe ne suffit pas pour un cas industriel où les décisions dépendent de sources internes, de contraintes de sécurité et de métriques opérationnelles. L’approche multi-agents, fondée sur ReAct, RAG, base de connaissances et statut serveur, apporte un gain mesurable.

Sur le benchmark principal, le système multi-agents obtient un score composite de **75,2/100**, contre **35,0/100** pour le LLM unique. Il réduit la latence de **63,7 %**, divise le coût par **7,91**, réduit les tokens de **88,3 %** et conserve un score sécurité de **10/10**. Ces résultats valident l’intérêt de l’architecture outillée.

La conclusion reste cependant prudente : les deux architectures échouent au gate de qualité, car le meilleur score moyen n’est que de **3,8/10** pour un seuil fixé à **7/10**. Le projet ne prouve donc pas qu’un agent BibOps est prêt à remplacer un support IT humain. Il prouve plutôt qu’un framework d’évaluation peut identifier objectivement les progrès, les faiblesses et les risques avant de parler de déploiement.

La Racing Arena complète cette conclusion en montrant que la sécurité des agents est une question d’architecture. Dans le scénario observé, l’équipe validée n’exécute aucune injection et ne fuit aucune stratégie, alors qu’une équipe ReAct exposée exécute 93 % des injections reçues. Ce résultat justifie l’intégration des tests adversariaux dans le cycle de benchmark.

Le travail accompli fournit donc une base solide :

- un agent IT traçable et outillé ;
- une évaluation multi-critères explicite ;
- des artefacts JSON et graphiques exploitables ;
- une suite de 552 tests unitaires passés ;
- une simulation adversariale pour analyser les risques agentiques.

Les prochaines étapes sont claires : augmenter le corpus et le nombre de tickets évalués, améliorer la qualité des prompts et du RAG, calibrer les jugements avec des experts humains, durcir la sécurité des endpoints et rendre la génération du rapport entièrement reproductible. Avec ces améliorations, BIBOPS peut devenir un outil de décision robuste pour comparer et gouverner des assistants LLM en environnement industriel.

Bibliographie

- [1] Fabio Perez and Ian Ribeiro. Ignore previous prompt : Attack techniques for language models. *arXiv preprint*, 2211.09527, 2022.
- [2] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct : Synergizing reasoning and acting in language models. *arXiv preprint*, 2210.03629, 2022.